

Boas-vindas e contexto: o que é um ORM?

Object-Relational Mapper — a ponte entre classes TypeScript e tabelas SQL

ORM (Object-Relational Mapper) traduz automaticamente entre **classes TypeScript** e **tabelas SQL**. Em vez de escrever SQL manualmente, você define classes decoradas e o ORM gera e executa as queries.

Sem ORM — SQL manual (node-postgres):

```
sem-orm.ts

// Escrito à mão em cada Repository
const { rows } = await db.query(
  'SELECT * FROM produtos WHERE id=$1',
  [id]
);

// Mapeamento manual:
return { id: rows[0].id, nome: rows[0].nome, ... };
```

Com TypeORM — classe mapeada:

```
com-orm.ts

@Entity()
export class Produto {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  nome: string;
}

// O TypeORM faz o resto automaticamente!
```

Boas-vindas e contexto: o que é um ORM?

Object-Relational Mapper — a ponte entre classes TypeScript e tabelas SQL

Por que usar TypeORM?



Produtividade

Menos código repetitivo — sem SQL manual para operações básicas.



Type Safety

Erros de tipo detectados em tempo de compilação, não em runtime.



Migrations

Controle de versão do schema — sem ALTER TABLE manual.



Multi-banco

PostgreSQL, MySQL, SQLite, SQL Server
— troca com um config.

Setup — Instalação e DataSource

Do zero ao TypeORM conectado ao PostgreSQL

terminal + tsconfig.json

1. Instalar dependências

```
npm install typeorm reflect-metadata pg
```

```
npm install -D @types/pg
```

2. tsconfig.json — habilitar decorators (obrigatório!)

Adicione/confirme estas duas opções:

```
"experimentalDecorators": true,
```

```
"emitDecoratorMetadata": true,
```

3. Importar reflect-metadata NO TOPO do index.ts

```
import 'reflect-metadata'; // DEVE ser o primeiro import!
```

⚠ Atenção ao tsconfig:

- experimentalDecorators: true — habilita @Entity, @Column...
- emitDecoratorMetadata: true — TypeORM precisa do metadata
- import 'reflect-metadata' SEMPRE o primeiro import do projeto
- synchronize: false em produção — use migrations

Setup — Instalação e DataSource

Do zero ao TypeORM conectado ao PostgreSQL

```
src/config/data-source.ts

// src/config/data-source.ts
import 'reflect-metadata';
import { DataSource } from 'typeorm';
import { Produto } from '../entities/Produto';

export const AppDataSource = new DataSource({
  type: 'postgres',
  host: process.env.DB_HOST ?? 'localhost',
  port: Number(process.env.DB_PORT) || 5432,
  username: process.env.DB_USER ?? 'postgres',
  password: process.env.DB_PASS ?? '',
  database: process.env.DB_NAME ?? 'api_produtos',
  entities: [Produto], // registrar todas as entidades
  migrations: ['src/migrations/*.ts'],
  synchronize: false, // NUNCA true em produção!
  logging: true,
});
```

```
src/index.ts

await AppDataSource.initialize()
  .then() {
    console.log('DB conectado!');
  })
  .catch((err) => {
    console.error(err);
  })
```

Entities e Decorators

A classe TypeScript que vira tabela no banco

Entity é uma classe TypeScript decorada com `@Entity()`. Cada propriedade decorada vira uma coluna. O TypeORM cria/gerencia a tabela automaticamente via migrations.

Decorators de coluna:

<code>@PrimaryGeneratedColumn()</code>	id SERIAL, auto-incremento
<code>@PrimaryGeneratedColumn('uuid')</code>	UUID gerado automaticamente
<code>@Column()</code>	Coluna simples (NOT NULL padrão)
<code>@Column({ nullable: true })</code>	Coluna que aceita NULL
<code>@Column({ unique: true })</code>	UNIQUE constraint
<code>@Column({ default: 0 })</code>	Valor padrão
<code>@Column('decimal', ...)</code>	Tipo numérico preciso
<code>@CreateDateColumn()</code>	DEFAULT NOW() no INSERT
<code>@UpdateDateColumn()</code>	Atualiza no UPDATE
<code>@DeleteDateColumn()</code>	Soft delete — nullable

```
src/entities/Produto.ts

// src/entities/Produto.ts
import {
  Entity, PrimaryGeneratedColumn, Column,
  CreateDateColumn, UpdateDateColumn
} from 'typeorm';

@Entity('produtos') // nome da tabela no banco
export class Produto {

  @PrimaryGeneratedColumn() // id SERIAL AUTO_INCREMENT
  id: number;

  @Column({ length: 150, unique: true })
  nome: string;
```

Entities e Decorators

A classe TypeScript que vira tabela no banco

Entity é uma classe TypeScript decorada com `@Entity()`. Cada propriedade decorada vira uma coluna. O TypeORM cria/gerencia a tabela automaticamente via migrations.

Continuação do código

```
src/entities/Produto.ts

// src/entities/Produto.ts

@Column('decimal', { precision: 10, scale: 2 })
preco: number;

@Column({ default: 0 })
estoque: number;

@Column({ default: true })
ativo: boolean;

@CreateDateColumn() // preenchido automaticamente no INSERT
criadoEm: Date;

@UpdateDateColumn() // atualizado automaticamente no UPDATE
atualizadoEm: Date;

}
```


Relacionamentos — @OneToMany, @ManyToOne, @ManyToMany

Definindo relações entre entidades com decorators

```
entities/Categoria.ts + Produto.ts

// src/entities/Categoria.ts
@Entity('categorias')
export class Categoria {
  @PrimaryGeneratedColumn() id: number;
  @Column() nome: string;

  // Uma categoria tem MUITOS produtos
  @OneToMany(() => Produto, (p) => p.categoria)
  produtos: Produto[];
}

// src/entities/Produto.ts - lado Many (FK aqui)
@ManyToOne(() => Categoria, (c) => c.produtos, {
  nullable: false,      // NOT NULL
  eager: false,         // não carrega automaticamente
})
@JoinColumn({ name: 'categoria_id' }) // nome da FK
categoria: Categoria;
```

Tipos de relacionamento:

@OneToOne

FK unique em uma das tabelas

Ex: *Usuário* ↔ *Perfil*

@OneToMany

FK na tabela Many, sem coluna aqui

Ex: *Categoria* → *Produtos*

@ManyToOne

FK nesta tabela (lado da coluna)

Ex: *Produto* → *Categoria*

@ManyToMany

Tabela pivot gerada pelo @JoinTable()

Ex: *Produto* ↔ *Tags*

Relacionamentos — @OneToMany, @ManyToOne, @ManyToMany

Definindo relações entre entidades com decorators

Continuação do código

```
entities/Categoria.ts + Produto.ts

// @ManyToMany — Tag em Produto
@ManyToMany(() => Tag)
@JoinTable() // cria tabela pivot automaticamente
tags: Tag[];
```

Tipos de relacionamento:

@OneToOne

FK unique em uma das tabelas

Ex: *Usuário* ↔ *Perfil*

@OneToMany

FK na tabela Many, sem coluna aqui

Ex: *Categoria* → *Produtos*

@ManyToOne

FK nesta tabela (lado da coluna)

Ex: *Produto* → *Categoria*

@ManyToMany

Tabela pivot gerada pelo @JoinTable()

Ex: *Produto* ↔ *Tags*



ATIVIDADE 1 | Modelagem com Entities

```
atividade-01.ts

// DESAFIO 1 - Entity Aluno
// src/entities/Aluno.ts
// Campos: id (gerado), nome (string), email (unique), ativo (boolean, default true)
// criadoEm (CreateDateColumn), atualizadoEm (UpdateDateColumn)

// DESAFIO 2 - Entity Disciplina
// src/entities/Disciplina.ts
// Campos: id, nome (unique), cargaHoraria (number), preco (decimal 10,2)

// DESAFIO 3 - Entity Matricula (tabela de junção com dados)
// src/entities/Matricula.ts
// Campos: id, notaFinal (decimal, nullable), situacao (string, nullable)
// Relações: @ManyToOne → Aluno, @ManyToOne → Disciplina

// DESAFIO 4 - Registrar no DataSource
// Adicione as 3 entities no array 'entities' do AppDataSource
entities: [Aluno, Disciplina, Matricula]

// DESAFIO 5 - Teste de compilação
npx tsc --noEmit // deve passar sem erros de tipo

// BONUS: adicione @Index() em email de Aluno e nome de Disciplina
```